F1 Race

DALI Multi-Agent System — Technical Documentation

Piccirilli Michael

February 24, 2026

Contents

1 Introduction **2**
1.1 DALI Overview . . . 2
1.2 Project Goals . . . 2

2 System Requirements **2**

3 Project Structure **3**

4 Agents **3**
4.1 Fixed Infrastructure Agents . . . 3
4.2 Car Agents (Dynamic) . . . 4

5 Dynamic Agent System **4**
5.1 Code Generation Pipeline . . . 4
5.2 Adding or Removing a Car . . . 4

6 Race Flow **5**
6.1 Startup Sequence . . . 5
6.2 Lap Round-Robin . . . 5

7 Timing System **5**

8 DALI Event Types **6**
8.1 External Events . . . 6
8.2 Internal Events . . . 6
8.3 Non-deterministic Track Events . . . 7
8.4 Safety Car Chain . . . 7

9 How to Run **7**
9.1 Native (WSL / Linux) . . . 7
9.2 Web Dashboard . . . 8
9.3 Dashboard Controls . . . 8

10 Docker Deployment **8**
10.1 Architecture . . . 8
10.2 Quickstart . . . 9

10.3 Environment Variables 9

11 DALI Syntax Reference **9**

12 Shutdown **10**

Abstract

This document describes the design, architecture, and implementation of a Formula 1 race simulation built on top of the DALI Multi-Agent System framework. Car agents are dynamically generated from a single agents.json configuration file — no agents are hardcoded in the codebase. A dedicated Pit-Wall agent coordinates the entire race flow and injects probabilistic track events (safety car deployment, heavy rain) at runtime. A real-time web dashboard provides live monitoring and manual control of the simulation.

1 Introduction

1.1 DALI Overview

DALI (Dynamic Agent Logic framework for Interaction) is a logic-based multi-agent system built on top of SICStus Prolog. Each agent is defined by a set of *external events* (reactions to incoming messages), *internal events* (proactive behaviours triggered by conditions), and standard Prolog clauses for knowledge representation and reasoning.

DALI agents run as independent SICStus processes that communicate via a shared message server running on a configurable TCP port (default: `3010`).

1.2 Project Goals

The F1 Race simulation was developed with the following goals in mind:

- Demonstrate the *reactive* and *proactive* capabilities of DALI agents in a realistic, event-driven scenario.
- Show how a fully *dynamic* agent roster can be maintained through code generation, avoiding any hardcoded agent names in the DALI source.
- Provide a real-time monitoring interface that reflects agent activity without modifying the agent logic.
- Package the entire stack as a reproducible **Docker Compose** application.

2 System Requirements

Component	Requirement
SICStus Prolog	4.6.0 at `/usr/local/sicstus4.6.0` (Linux/WSL)
Shell	`bash`, `tmux`
Python	3.8+ (for `generate_agents.py` and dashboard)
Flask	Auto-installed in a local venv by `ui/run.sh`
Docker	Optional; required only for the Docker-based deployment

3 Project Structure

```
f1_race/
+-- agents.json            <- Single source of truth for car
   agents
+-- generate_agents.py     <- Generates DALI files from agents.
   json
+-- startmas.sh            <- Launch script (Linux/WSL)
+-- startmas.bat           <- Launch script (Windows)
+-- docker-compose.yml     <- Docker Compose (mas + ui containers
   )
+-- .env.example           <- Template for SICSTUS_PATH
+-- docker/
|   +-- setup.sh           <- Auto-detects SICStus, writes .env
|   +-- mas/Dockerfile
|   \-- ui/Dockerfile
+-- ui/
|   +-- dashboard.py       <- Flask backend
|   +-- run.sh             <- Creates venv + launches dashboard
|   +-- requirements.txt
|   \-- static/
|       +-- app.js         <- Frontend (syncConfig every 5 s)
|       \-- index.html
+-- mas/
|   +-- instances/         <- Agent instance files
|   \-- types/             <- Agent type (behaviour) files
+-- conf/                  <- DALI communication configuration
+-- build/                 <- Runtime binaries (auto-generated)
+-- work/                  <- Runtime working files (auto-
   generated)
\-- log/                   <- Agent logs (auto-generated)
```

4 Agents

4.1 Fixed Infrastructure Agents

The following agents are always present regardless of the car roster:

Agent	Instance file	Type	Role
`semaphore`	`instances/semaphore.txt`	`semaphoreType`	Collects `ready` signals, runs the F1 lights sequence, fires `start_race`
`pitwall`	`instances/pitwall.txt`	`pitWallType`	Race coordinator: manages lap order, lap times, standings, track events
`safety_car`	`instances/safety_car.txt`	`safetyCarType`	Reacts to deploy/recall; forwards `sc_recalled` and participates in the green-flag chain

4.2 Car Agents (Dynamic)

Car agents are *never hardcoded*. The entire roster is driven by `agents.json`. The default configuration defines four cars:

ID	Team	Driver	Car model
`ferrari`	Ferrari	Leclerc	SF-24
`mclaren`	McLaren	Norris	MCL38
`redbull`	Red Bull	Verstappen	RB20
`mercedes`	Mercedes	Hamilton	W15

Each car entry in `agents.json` also carries UI metadata (`color`, `border`, `label`) used by the dashboard.

5 Dynamic Agent System

5.1 Code Generation Pipeline

The `generate_agents.py` script is the single point of truth for all generated files:

```
agents.json
    +-> generate_agents.py
          +-- mas/instances/{id}.txt        (one per car)
          +-- mas/types/{id}Car.txt         (one per car)
          +-- mas/types/pitWallType.txt     (round-robin over all
              cars)
          +-- mas/types/semaphoreType.txt   (waits N_cars + 2
             ready)
          \-- mas/types/safetyCarType.txt
```

`generate_agents.py` is invoked automatically at each MAS startup by `startmas.sh`. It is *smart about regeneration*:

Situation	Behaviour
`agents.json` unchanged	Skips regeneration — no files written
New car added	Full regeneration
Car removed	Stale `{id}.txt` / `{id}Car.txt` deleted, then full regeneration
`--force` flag	Always regenerates unconditionally

5.2 Adding or Removing a Car

Edit `agents.json` and add a new entry to the `cars` array:

```
{
  "total_laps": 5,
  "cars": [
    { "id": "ferrari",   "team": "Ferrari",  "car_model": "SF
       -24",
```

```
      "driver": "Leclerc", "label": "Ferrari SF-24",
      "color": "#180505", "border": "#cc2200" },
    { "id": "myclubcar", "team": "My Club",  "car_model": "X1",
      "driver": "Rossi",   "label": "Club X1",
      "color": "#001020", "border": "#00aaff" }
  ]
}
```

Then run `bash startmas.sh`. All DALI files and the dashboard update automatically.

6 Race Flow

6.1 Startup Sequence

1. Every agent (all cars, pit wall, safety car) sends a `ready` message to the semaphore.
2. The semaphore waits until it has collected exactly $N_{\text{cars}} + 2$ ready signals.
3. The semaphore runs the F1 lights sequence (5 lights on, 1 s each; 2 s pause; lights out) and sends `start_race` to `Car[0]`.

6.2 Lap Round-Robin

Once the race starts, laps proceed in a strict round-robin over the ordered car list:

1. `Car[i]` receives `lap_go_{id}` from the pit wall.
2. `Car[i]` asserts its lap status and sends `lap_done_{id}` to the pit wall.
3. The pit wall adds a random lap time, optionally fires a track event, and sends `lap_go_{id+1}` to the next car.
4. After the last car of a lap completes:
   - Lap counter increments.
   - Standings are printed.
   - If laps remain: `random_track_event` is rolled, then `lap_go` is sent to `Car[0]`.
   - If race complete: `declare_winner` is called and `race_end` is broadcast.

At any point a car's *internal events* (`engine_failureI` or `push_lapI`) can fire autonomously and notify the pit wall.

7 Timing System

The winner is the car with the lowest total accumulated time. Time is measured in seconds.

Event	**Time change**
Each lap	+ `random`(60, 90) seconds

Event	Time change
Pit stop	+25 s
Safety car (20 % per lap)	+10 s to *all* cars
Heavy rain (20 % per lap)	+5 s to *all* cars
Push lap (internal event, ≈10 %)	−3 s
Engine failure / DNF (≈0.2 %)	`time` = 9999 s (race ends)

8 DALI Event Types

8.1 External Events

External events (`nameE:> Body.`) are reactive: they fire when a message named `name` arrives from another agent.

Event	Description
`start_raceE`	Lights-out signal from semaphore; asserts `race_started`
`lap_go_{id}E`	Pit wall tells car to start its next lap
`lap_done_{id}E`	Car finishes a lap; pit wall adds time, rolls track event
`pit_done_{id}E`	Car finishes pit stop; pit wall adds +25 s
`{id}_engine_failureE`	DNF notification; sets effective time to 9999
`{id}_push_lapE`	Fastest-lap bonus; subtracts 3 s
`race_endE`	Broadcast at end of race; asserts local `race_over`
`rain_warningE`	Cosmetic notification (ignored if `race_over`)
`safety_car_deployedE`	Car receives safety-car notification
`green_flagE`	Sent after safety car recalled; car reacts with a push message
`retire_{id}E`	Car parks; cosmetic DNF notification

8.2 Internal Events

Internal events (`nameI:> Body.`) are proactive: they fire autonomously whenever the associated condition clause `name :- Cond.` succeeds.

Listing 1: Generated internal events (per-car)

```
% Engine failure -- fires at most once, ~0.2% probability per cycle
engine_failure_{id} :-
    race_started,
    \+ race_over,
    \+ engine_failure_{id}_fired,
    random(0, 1000, R), R < 2.

engine_failure_{id}I:>
    assert(engine_failure_{id}_fired),
    send_m(pitwall, send_message({id}_engine_failure, {id})).
```

```


% Push lap -- fires at most once per lap, ~10% probability
push_lap_{id} :-
    race_started,
    \+ race_over,
    \+ push_lap_{id}_fired,
    random(0, 100, R), R < 10.

push_lap_{id}I:>
    assert(push_lap_{id}_fired),
    send_m(pitwall, send_message({id}_push_lap, {id})).
```

The `push_lap_{id}_fired` flag is retracted at the start of each new lap (`lap_go` handler), so the event can fire once per lap.

8.3 Non-deterministic Track Events

The pit wall rolls `random_track_event/0` after each car completes a lap (never on the last lap, at most once per lap via the `track_event_this_lap` flag):

Listing 2: random_track_event/0 in pitWallType

```
random_track_event :-
    if(track_event_this_lap,
       true,
       ( random(0, 10, R),
         if(R < 2,
            /* 20%: SAFETY CAR -- +10s to all cars */,
         if(R < 4,
            /* 20%: HEAVY RAIN -- +5s to all cars */,
            true)) )).
```

8.4 Safety Car Chain

The safety car involves a three-step message chain:

1. Pit Wall detects `R < 2` ⇒ asserts `track_event_this_lap`, sends `deploy` to Safety Car and `safety_car_deployed` to all cars.
2. At end-of-lap the pit wall sends `recall` to Safety Car ⇒ Safety Car retracts `sc_active` and sends `sc_recalled` back to pit wall.
3. Pit wall receives `sc_recalled` ⇒ prints "GREEN FLAG!" and sends `green_flag` to all cars.

9 How to Run

9.1 Native (WSL / Linux)

```
cd DALI/Examples/f1_race
bash startmas.sh
```

`startmas.sh` performs the following steps:

1. Kills any stale SICStus / tmux processes on port 3010.
2. Runs `generate_agents.py` to sync DALI files with `agents.json`.
3. Builds agent binaries and launches all agents inside a single `tmux` session named `f1_race`.

9.2 Web Dashboard

In a second terminal:

```
cd DALI/Examples/f1_race
bash ui/run.sh
```

`run.sh` creates a local Python venv (`ui/.venv`) on the first run and installs Flask automatically. Subsequent launches skip `pip install` via a stamp file (`.venv/.installed_stamp`) and start in approximately 1 second.

Open `http://localhost:5000` in a browser. The dashboard shows all agent panes side-by-side with real-time auto-scrolling. If `agents.json` is modified while the dashboard is open, it auto-detects the change within 5 seconds and rebuilds the grid without a page reload.

9.3 Dashboard Controls

UI element	Function
Restart MAS	Kills SICStus + tmux session, reruns `startmas.sh`
Deploy SC	Sends deploy message to safety car immediately
Recall SC	Recalls the safety car
Agent / Command bar	Sends an arbitrary Prolog command to any agent pane
Pin button (↓)	Toggles auto-scroll for that pane
Clear button (×)	Clears pane output
Minimise button (−)	Minimises pane to tray

10 Docker Deployment

10.1 Architecture

The Docker Compose setup runs two containers that share a tmux socket volume, allowing the dashboard to read agent panes with zero modifications to the agent logic.

```
+-----------------------------------+
   +-------------------------------+
```

```
|  mas  (ubuntu:22.04)              |   |  ui  (python:3.11-slim)
        |
|  startmas.sh -> tmux f1_race      |   |  dashboard.py (Flask
   :5000)  |
|  SICStus mounted from host (ro)   |   |  reads tmux panes via
   volume |
+-----------------------------------+
   +------------------------------+
                  tmux_sock volume (/tmp/tmux-shared)
```

Startup order (managed by Docker healthchecks):

1. `ui` starts immediately and is healthy once `/api/config` responds.
2. `mas` waits for `ui` to be healthy, then runs `startmas.sh`.

10.2 Quickstart

```
cd DALI/Examples/f1_race

# 1. Auto-detect SICStus and write .env  (run once)
bash docker/setup.sh

# 2. Build and start
docker compose up --build -d

# 3. Open dashboard
open http://localhost:5000
```

10.3 Environment Variables

Variable	Default	Description
`SICSTUS_PATH`	`/usr/local/sicstus4.6.0`	Host path to SICStus install, mounted read-only into the `mas` container

11 DALI Syntax Reference

The following table summarises all DALI syntax constructs used in this project.

Syntax	Meaning	Example
`nameE:> Body.`	React to external event `name`	`start_raceE:> write('Go!').`
`nameI:> Body.`	Fire internal event when `name` holds	`push_lap_ferrariI:> send_m(...).`
`name :- Cond.`	Condition for internal event `nameI`	`push_lap_ferrari :- race_started, ...`

Syntax	Meaning	Example
`messageA(ag,msg)`	Send message (top-level clause)	`messageA(pitwall, lap_done).`
`send_m(ag,msg)`	Send message (safe inside `if/3`)	`send_m(safety_car, deploy).`
`random(Lo,Hi,R)`	Random integer `Lo` $\leq R <$ `Hi`	`random(0, 10, R).`
`if(Cond,Then,Else)`	Conditional	`if(R < 5, act, true).`
`\+ Goal`	Negation-as-failure	`\+ race_over`
`:- Goal.`	Directive (runs at load time)	`:- write('Agent ready!').`
`keysort(+P,-S)`	Sort `Key-Value` pairs by key	`keysort([3-b,1-a],S).`

12 Shutdown

```
# Stop the MAS
tmux kill-session -t f1_race

# Stop the Flask dashboard
# Ctrl+C in the dashboard terminal

# Stop Docker
docker compose down
```